

Semantic Search → **Moment** Search

Turning a 2-hour system-design lecture into cited, click-to-play moments.

Jessie Lai · reference: [Moment_RAG/](#) · video [C842vFY5kRo](#)

The video — and why

- **System Design Course (YouTube C842vFY5kRo)** — a **125.4-minute** system-design course (Hamza-recommended fundamentals).
- **Why this one:** long and topic-dense is exactly where *document* RAG fails — "where does he explain sharding?" should return a 90-second **moment**, not a paragraph that happens to mention it.
- Corpus: **3,173 cues · 20,861 words · 24,321 tokens** · single speaker · auto-captions.

What "Moment RAG" is

Intelligence on the **query** side; retrieval resolves to **moments**, not documents.

```
user question
|
├ self-query → hard filters (topic / section_type)
├ decompose → 2-4 sub-queries (one question → facets)
|
├ HYBRID retrieve in Qdrant, per sub-query:
|   dense kNN · BM25 sparse · HyDE-question multivector
|   └ fused with RRF (server-side)
|
├ cross-encoder rerank (top-50) → which moment wins
└ cited synthesis → [n] deep-links into YouTube at the exact second
```

HyDE at ingest, not query time: each chunk is pre-tagged with the questions it answers; those are embedded as a 3rd branch, so a user question can match a stored *question*. Keeps query latency low.

Part 1 — Baseline semantic search

The control: standard document RAG.

```
transcript → fixed 256-tok windows (25% stride) → OpenAI embed  
→ cosine kNN top-k → stuff into LLM → answer
```

- One dense signal, one pass. No rewriting, no reranking, no filters.
- Retrieval unit = a **coarse 256-token window** chosen by token count — blind to where a topic starts or ends.
- Result on this video: **145 windows · avg 255 tokens · avg 75s span**.

Part 2 — Moment RAG (full port)

INGESTION (BUILD INTELLIGENCE INTO THE INDEX)

- **Semantic chunking**: 30-word units → bge-small embeddings → cut at 90th-pctile cosine-distance spikes → merge to [50,256] tokens.
- **LLM enrichment** (gpt-4o-mini, 8-way): hypothetical questions, keywords, acronyms, gist, section_type, **topic**.
- **Embed** chunk text + questions @ text-embedding-3-large/1024.

Result: 125 semantic moments · avg 195 tokens · avg 60s span.

RETRIEVAL (QDRANT HYBRID)

- 3 named vectors: dense (cosine) · BM25 (IDF sparse) · questions (multivector MaxSim).
- RRF fusion across every (branch × sub-query), **server-side**.
- Cross-encoder rerank (jina-tiny) → timestamped moments + deep-links.

Three single-video adaptations

The reference targets a multi-video podcast corpus; one long lecture needs deliberate changes.

1. **Semantic chunking** replaces fixed windows — a moment tracks a topic boundary, not a token count.
2. **content_labels** → **per-chunk topic** . Self-query across videos is degenerate with one source, so enrichment tags each chunk with a system-design topic — restoring cross-*topic* disambiguation *within* the lecture.
3. **Rollup cites moments, not videos**. Video-level rollup collapses to one item, so we cite the reranked **moments** directly.

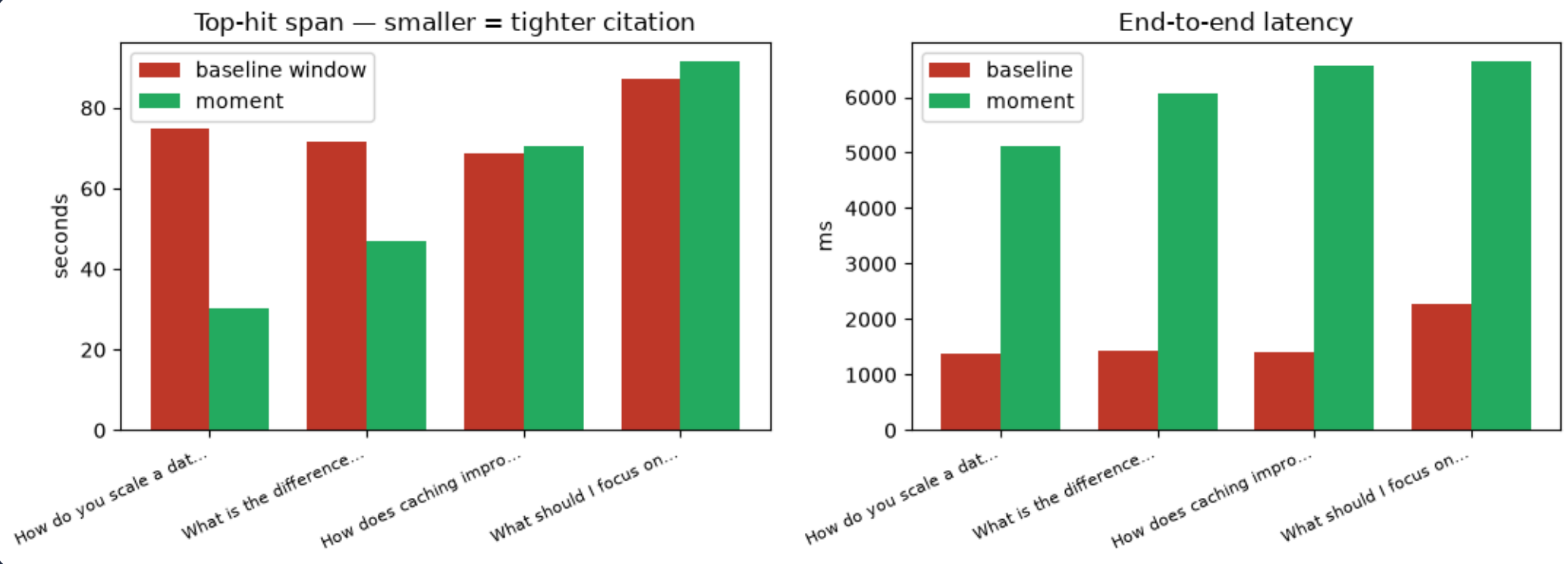
Comparison — real numbers

METRIC	BASELINE	MOMENT RAG
retrieval unit	256-tok window	semantic moment
chunks indexed	145	125
avg top-hit span	76s	60s (~21% tighter)
retrieval signals	1 (dense)	3 (dense+bm25+Q)
decomposition	no	yes (4.0 sub-q)
rerank / self-query	no	yes
clickable moment link	no	yes (YouTube &t=)
avg latency	2,209 ms	6,082 ms

SCORED EVAL — HELD-OUT TEST SPLIT (6 QS, DISJOINT FROM THE 6 TUNING QS)

METRIC	BASELINE	MOMENT RAG
LLM judge (1–5, Claude Haiku)	1.5	2.0
region-accuracy (answer lands in the right topical region)	0.63	0.87
citation-accuracy (cites point to the right region)	0.63	0.88
context tokens	511	569 (+11%)
latency	1,439 ms	6,236 ms (~4.3×)
cost / query	\$0.00015	\$0.00033 (~2.2×)

Both pipelines degrade dev→test (no overfit to the tuning knob). Tuned knobs: baseline k=4, moment size=4. Judge = Claude Haiku (claude-haiku-4-5) via the direct Anthropic API — a different vendor/model from the gpt-4o-mini synthesizer, so no self-preference. Haiku grades strictly (low absolute scores); the moment-RAG lift is what matters. See limits.



Example — "How does caching improve performance and what are the tradeoffs?"

BASELINE (COARSE, NO LINKS)

Caching improves performance primarily by storing frequently accessed data in a way that allows for faster retrieval, reducing the need to repeatedly fetch data from slower sources like databases. Specifically, application-level caching can be leveraged to enhance performance beyond just HTTP caching [2].

However, there are tradeoffs associated with caching. One limitation is that it can lead to resource constraints, as there is a cap on how much data can be stored, especially in a single server setup [3]. Additionally, caching can introduce complexity in ensuring data consistency, as the cached data may become stale if not managed properly.

Overall, while caching significantly boosts performance by speeding up data access, it requires careful management to mitigate potential downsides such as resource limits and data consistency issues.

MOMENT RAG (CITED MOMENTS)

Caching Enhances Performance by Reducing Latency

TL;DR Caching significantly improves performance by storing frequently accessed data closer to the user, minimizing retrieval times. However, it introduces tradeoffs such as data staleness and increased complexity in data management.

MECHANISM OF CACHING

Caching works by temporarily storing copies of data in locations that can be accessed more quickly than the original source. This mechanism reduces the load on databases and speeds up response times for end-users. The instructor emphasizes that appropriate caching strategies are essential for efficient API design, especially when handling large datasets, to avoid overwhelming the system with requests for the same data repeatedly [1].

EXAMPLE OF CACHING IN APIS

Consider an API that serves a large number of posts. If every request retrieves thousands of posts from the database, this can lead to performance bottlenecks. Instead, implementing pagination along with caching ensures that only a limited number of posts are fetched at a time, drastically improving performance and user experience. By caching responses to common queries, the system can serve repeated requests without hitting the database each time, thereby enhancing speed [1].

TRADEOFFS OF CACHING

While caching improves performance, it also has its downsides. One significant drawback is the potential for data to become stale, as cached data may not reflect the most current state of the database. This can lead to inconsistencies if the underlying data changes after it has been cached. Additionally, implementing caching can complicate the architecture of the system, requiring careful management of cache invalidation and updates to ensure data integrity [1].

Bottom line: Effective caching strategies can dramatically improve performance, but must be carefully managed to mitigate risks associated with stale data and increased system complexity.

[1] 40:52 · api-design

[2] 02:00 · fundamentals

[3] 37:28 · api-design

[4] 38:18 · api-design

[5] 123:52 · security

Findings, limits & what's next

FINDINGS (SCORED, TEST SPLIT)

- **Higher-quality answers:** judge 2.0 vs 1.5 (Claude Haiku, strict grader); region-accuracy 0.87 vs 0.63; citation-accuracy 0.88 vs 0.63 — moment RAG lands in the right topical region and cites it more reliably.
- **Cost = latency + spend:** ~4.3× latency (6,236 vs 1,439 ms), ~2.2× cost, +11% context tokens — the price of decomposition + 3 fused signals + rerank. Worth it for "find & explain the moment"; overkill for FAQ lookup.
- **No overfit:** both degrade dev→test (region-acc 0.92→0.87 moment, 0.75→0.63 baseline), so the win isn't an artifact of tuning the retrieval knob on the same questions.

LIMITS (HONEST)

- **Ground truth is LLM-labeled:** the topical regions were segmented by gpt-4o-mini, not objective section boundaries — softer than a numbered-chapter gold set.
- **Single judge, small n:** the judge is Claude Haiku (claude-haiku-4-5) via the direct Anthropic API — a different vendor/model from the gpt-4o-mini synthesizer, so there's no same-model self-preference. But it's still one judge on a 6-question test set, and it grades strictly (low absolute scores), so treat the numbers as directional.
- **Tuning leakage risk:** retrieval knob tuned on 6 dev Qs, reported on 6 disjoint test Qs; small n, so numbers are directional.
- **Purity ≠ completeness:** on the multi-region enumeration Q, moment RAG is both purer (region-acc 1.00 vs 0.50) and more complete (topic-recall 0.75 vs 0.25) — but it still misses a quarter of the required topics, so enumeration ("list every strategy") remains the harder case.

Knowledge graph (requested extension): enrichment already extracts per-moment keywords/topics/acronyms — the raw material. Next: `(concept)-[discussed_at]->(moment)` and `(concept)-[related_to]->(concept)` edges (co-occurrence + an LLM relation pass) → graph-aware retrieval and a browsable concept map. Scoped, not built.